Use safe-exceptions
safe-exceptions: https://hackage.haskell.org/package/safe-exceptions

In particular, safe-exceptions handles **masking** as expected.

e.g.

```
start :: IO ()
done :: IO ()
process :: IO ()

-- BAD. normal bracket_
import Control.Exception
bracket_
  start
  done                                  -- BAD. done may be
interrupted by async exception and never finish
  process


-- Good. safe-exceptions bracket_
import Control.Exception.Safe
bracket_
 start
 done                                   -- GOOD. done is never
interrupted
 process                                -- same as
(uninterruptibleMask_ done)
```

MonadExcept (aka MonadError, ExceptT, ErrorT) vs **MonadCatch** (catch/throw)
Exception best practices: https://www.fpcomplete.com/blog/exceptions-best-practices-haskell/

Do NOT use **MonadExcept (ExceptT, ErrorT**)
1. Error type is **fixed**
e.g.
ExceptT MyError IO a only handles MyError

2. **Multiple error handling**: in many cases, the **base layer** inherently throws errors e.g. **IO**.
So we'll need to handle error for both **MonadExcept** and **IO**.

e,g,
Bad. Need 2 error handlers

```
import Control.Monad.Reader
import Control.Monad.Trans.Except (ExceptT (..), runExceptT)
import Control.Exception.Safe

runApp :: r -> ReaderT r (ExceptT MyException IO) a -> IO a
runApp env app = do
```

```
  let io = runExceptT $ runReaderT app env
  result <- catch io (handleErr @SomeException)
    -- handle error for IO
  case result of
    -- handle error for ExceptT
    Left e -> handleErr e
    Right value -> return value
  where
    handleErr :: Exception e => e -> IO a
    handleErr e = print e >> throwM e
```

Good

```
import Control.Monad.Reader
import Control.Exception.Safe

runApp2 :: r -> ReaderT r IO a -> IO a
runApp2 env app = catch (runReaderT app env) handleErr
  where
    handleErr e@(SomeException _) = print e >> throwM e
```

==========